

• Buzzer

```
#define TRIG_PIN 6
```

```
#define ECHO_PIN 7
```

```
#define BUZZER 9
```

```
long duration;
```

```
float distance;
```

```
void setup() {
```

```
    Serial.begin(9600);
```

```
    pinMode(TRIG_PIN, OUTPUT);
```

```
    pinMode(ECHO_PIN, INPUT);
```

```
    pinMode(BUZZER, OUTPUT);
```

```
}
```

```
void loop() {
```

```
    // --- The Trigger Pulse ---
```

```
    digitalWrite(TRIG_PIN, LOW);
```

```
    delayMicroseconds(2);
```

```
    digitalWrite(TRIG_PIN, HIGH);
```

```
    delayMicroseconds(10);
```

```
digitalWrite(TRIG_PIN, LOW);

// --- Measurement ---
duration = pulseIn(ECHO_PIN, HIGH);
distance = duration * 0.034 / 2;

// --- The Frequency Change Logic ---
if (distance > 0 && distance < 50) {
    // Map distance (2-50cm) to Frequency (2000Hz - 100Hz)
    // As distance decreases, frequency increases
    int freq = map(distance, 2, 50, 2000, 100);
    tone(BUZZER, freq);
} else {
    noTone(BUZZER);
}

delay(100);
}
```

•SDMCET ON LCD

```
#include <LiquidCrystal.h>
```

```
// RS, EN, D4, D5, D6, D7
```

```
LiquidCrystal lcd(6, 7, 5, 4, 3, 2);
```

```
void setup() {
```

```
  lcd.begin(16, 2); // Initialize 16x2 display
```

```
  lcd.clear();      // Start with a clean screen
```

```
}
```

```
void loop() {
```

```
  // 1. Line One
```

```
  lcd.setCursor(0, 0);    // Move to Column 0, Row 0
```

```
  lcd.print("SDMCET");
```

```
  // 2. Line Two
```

```
  lcd.setCursor(0, 1);    // Move to Column 0, Row 1
```

```
  lcd.print("Aishwarya");
```

```
  delay(2000);           // Wait for 2 seconds
```

```
}
```

• UART0

```
#include <lpc214x.h>
```

```
// Function Prototype
```

```
void uart_interrupt(void) __irq;
```

```
unsigned char rx_Flag = 0x00;
```

```
unsigned char tx_Flag = 0x00;
```

```
int main(void) {
```

```
    // 1. Pin Selection: P0.0 as TXD0 and P0.1 as RXD0
```

```
    PINSEL0 = 0x00000005;
```

```
    // 2. UART0 Line Control: 8-bit, 1 stop bit, Enable DLAB
```

```
U0LCR = 0x83;
```

```
// 3. Baud Rate Setup (9600 bps @ 12MHz PCLK)
```

```
U0DLM = 0x00;
```

```
U0DLL = 0x61;
```

```
// 4. Disable DLAB to access Data/Interrupt registers
```

```
U0LCR = 0x03;
```

```
// 5. Enable RX and THRE (Transmit) Interrupts
```

```
U0IER = 0x03;
```

```
// 6. VIC (Vectored Interrupt Controller) Setup
```

```
VICVectAddr0 = (unsigned long)uart_interrupt; // Set address
```

```
VICVectCntl0 = 0x20 | 6; // 0x20 enables slot, 6 is UART0
```

```
ID
```

```
VICIntEnable = 0x00000040; // Enable UART0 interrupt (bit  
6)
```

```
while (1) {
```

```
    // Wait for a character to be received (flag set in ISR)
```

```
    while (rx_Flag == 0x00);
```

```
    rx_Flag = 0x00; // Reset flag
```

```

    // Wait for transmission to complete
    while (tx_Flag == 0x00);
    tx_Flag = 0x00; // Reset flag
}
}

// Interrupt Service Routine (ISR)
void uart_interrupt(void) __irq {
    unsigned int iid;

    iid = U0IIR;      // Read Interrupt Identification Register
    iid = (iid >> 1) & 0x07; // Extract bits 3:1

    if (iid == 0x02) { // THRE (Transmit Holding Register Empty)
        tx_Flag = 0xFF;
    }

    else if (iid == 0x04) { // RDA (Receive Data Available)
        U0THR = U0RBR; // Echo: Send received byte back immediately
        rx_Flag = 0xFF;
    }

    VICVectAddr = 0x00; // Signal end of interrupt to VIC
}

```

•Triangular wave

```
#include <lpc21xx.h>

int main() {
    unsigned long int temp;
    unsigned int i;
    IO0DIR = 0x00FF0000; // Pins P0.16 to P0.23 as Output

    while(1) {
        // Ramp Up
        for (i = 0; i <= 0xFF; i++) {
            IO0PIN = (i << 16); // Shift value to match output pins
        }
        // Ramp Down
        for (i = 0xFF; i > 0; i--) {
            IO0PIN = (i << 16);
        }
    }
}
```

•Sine Wave

```
#include <lpc21xx.h>
```

```
// 49 values representing one sine cycle
```

```
unsigned char sinetab[49] = { 0x80, 0x90, 0xA1, 0xB1, 0xC0, ... }; //
```

(Keep your values)

```
int main() {
```

```
    PINSEL0 = 0x00000000;
```

```
    IO0DIR = 0x00FF0000;
```

```
    int count;
```

```
    while(1) {
```

```
        for (count = 0; count < 49; count++) {
```

```
            IO0PIN = (unsigned long)sinetab[count] << 16;
```

```
        }
```

```
    }
```

```
}
```


• Square Wave

```
#include <lpc21xx.h>
```

```
void delay(void);
```

```
int main() {
```

```
    IO0DIR = 0x00FF0000; // P0.16-P0.23 as output
```

```
    while(1) {
```

```
        IO0PIN = 0x00FF0000; // All pins HIGH
```

```
        delay();
```

```
        IO0PIN = 0x00000000; // All pins LOW
```

```
        delay();
```

```
    }
```

```
}
```

```
void delay(void) {
```

```
    unsigned int i;
```

```
    for(i = 0; i <= 95000; i++); // Busy wait
```

```
}
```

•7 SEGMENT

```
#include <lpc214x.h>
```

```
unsigned int delay;
```

```
unsigned int switchcount = 0;
```

```
// 7-segment patterns for 0-F (Corrected Hex for Port 0, Bits 16-23)
```

```
unsigned int Disp[16] = {
```

```
    0x003F0000, 0x00060000, 0x005B0000, 0x004F0000,
```

```
    0x00660000, 0x006D0000, 0x007D0000, 0x00070000,
```

```
    0x007F0000, 0x006F0000, 0x00770000, 0x007C0000,
```

```
    0x00390000, 0x005E0000, 0x00790000, 0x00710000
```

```
};
```

```
#define ALLDISP 0x0F000000 // Selection pins P0.24 to P0.27
```

```
#define DATAPORT 0x00FF0000 // Data pins P0.16 to P0.23
```

```
int main(void) {
```

```
    PINSEL0 = 0x00000000;
```

```
    PINSEL1 = 0x00000000;
```

```
    IODIR0 = ALLDISP | DATAPORT; // Set all as outputs
```

```

while(1) {
    IOSET0 = ALLDISP;    // Select the displays
    IOCLR0 = DATAPORT;   // Clear previous digit
    IOSET0 = Disp[switchcount]; // Display current number

    // Button Check (Assuming active low button on P1.23)
    if (!(IOPIN1 & 0x00800000)) {
        for(delay=0; delay<1000000; delay++); // Debounce delay
        switchcount++;

        if (switchcount >= 16) {
            switchcount = 0;
        }
    }
}

```

• 7 segment using sensor

```
#include <ShiftRegister74HC595.h>
```

```
// Ultrasonic sensor pins
```

```
const int trigPin = 6;
```

```
const int echoPin = 7;
```

```
// Buzzer pin
```

```
int buzzer_pin = 9;
```

```
// Shift register setup (2 chained registers)
```

```
// Parameters: (number of registers, data pin, clock pin, latch pin)
```

```
ShiftRegister74HC595 sr(2, 12, 10, 11);
```

```
// 7-segment encoding for digits 0-9
```

```
uint8_t numberB[] = {
```

```
    0b00111111, // 0
```

```
    0b00000110, // 1
```

```
    0b01011011, // 2
```

```
    0b01001111, // 3
```

```
0b01100110, // 4
0b01101101, // 5
0b01111101, // 6
0b00000111, // 7
0b01111111, // 8
0b01101111 // 9
};

long duration;
int distance;

void setup() {
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  pinMode(buzzer_pin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  // Trigger ultrasonic pulse
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
```

```
delayMicroseconds(10);
digitalWrite(trigPin, LOW);

// Read echo
duration = pulseIn(echoPin, HIGH);

// Calculate distance in cm
// Equation: distance = (time * speed of sound) / 2
distance = duration * 0.034 / 2;

// Limit to two digits (0-99)
if (distance > 99) {
    distance = 99;
}

// Split into tens and ones
int tens = distance / 10;
int ones = distance % 10;

// Display on 7-segment
uint8_t pinValues[] = { numberB[tens], numberB[ones] };
sr.setAll(pinValues);
```

```
// Buzzer alert
if (distance < 5) {
    digitalWrite(buzzer_pin, HIGH);
} else {
    digitalWrite(buzzer_pin, LOW);
}

// Debug output
Serial.print("Distance: ");
Serial.println(distance);

delay(200);
}
```

• 7 segment with keypad

[3:34 pm, 26/5/2026] Sameet: #include <ShiftRegister74HC595.h>

#include <Keypad.h>

const byte numRows = 4;

const byte numCols = 4;

char keymap[numRows][numCols] = {

{ '0', '1', '2', '3' },

{ '4', '5', '6', '7' },

{ '8', '9', 'a', 'b' },

{ 'c', 'd', 'e', 'f' }

};

byte rowPins[numRows] = { 6, 7, 8, 9 };

byte colPins[numCols] = { 2, 3, 4, 5 };

Keypad myKeypad = Keypad(makeKeymap(keymap), rowPins, colPins, numRows, numCols);

ShiftRegister74HC595 sr(2, 12, 10, 11);

// Segment encoding for 0-9


```
uint8_t numberB[] = {  
    0b00111111, 0b00000110, 0b01011011, 0b01001111,  
    0b01100110, 0b01101101, 0b01111101, 0b00000111,  
    0b01111111, 0b01101111  
};
```

// Segment encoding for a-f

```
uint8_t hexvalues[] = {  
    0b01011111, 0b01111100, 0b00111001, 0b01011110,  
    0b01111001, 0b01110001  
};
```

```
void setup() {  
    Serial.begin(9600);  
}
```

```
void loop() {  
    char keypressed = myKeypad.getKey();  
  
    if (keypressed != NO_KEY) {  
        Serial.print("Keypressed: ");  
        Serial.println(keypressed);  
    }
```

```
if (keypressed >= '0' && keypressed <= '9') {  
    int i = keypressed - '0';  
    uint8_t pinValues[] = {numberB[i], numberB[i]};  
    sr.setAll(pinValues);  
}  
else if (keypressed >= 'a' && keypressed <= 'f') {  
    int i = keypressed - 'a';  
    uint8_t pinValues[] = {hexvalues[i], hexvalues[i]};  
    sr.setAll(pinValues);  
}  
delay(1000);  
}  
}
```

•7 segment without keypad

```
#include <ShiftRegister74HC595.h>
```

```
ShiftRegister74HC595 sr(2, 12, 10, 11);
```

```
uint8_t numberB[] = {
```

```
    0b00111111, 0b00000110, 0b01011011, 0b01001111,
```

```
    0b01100110, 0b01101101, 0b01111101, 0b00000111,
```

```
    0b01111111, 0b01101111
```

```
};
```

```
void setup() {
```

```
    // Setup is empty in your notes
```

```
}
```

```
void loop() {
```

```
    for (int i = 0; i < 10; i++) {
```

```
        for (int j = 0; j < 10; j++) {
```

```
            uint8_t pinValues[] = {numberB[i], numberB[j]};
```

```
            sr.setAll(pinValues);
```

```
            delay(1000);
```

```
}  
}  
}
```

•Temperature on serial monitor

```
// DISPLAY DHT VALUE ON SERIAL MONITOR
```

```
#include <dht.h>
```

```
dht DHT;
```

```
#define DHT11_PIN 4
```

```
void setup() {
```

```
  Serial.begin(9600);
```

```
}
```

```
void loop() {
```

```
  int chk = DHT.read11(DHT11_PIN);
```

```
  Serial.print("Temperature : ");
```

```
  Serial.println(DHT.temperature);
```

Serial.print("%"); // Note: You usually use 'C' for temp, but following
your notes

Serial.print("Humidity : ");

Serial.println(DHT.humidity);

Serial.print("%");

delay(2000);

}

•Data on LCD

```
// DISPLAY DHT VALUE ON LCD
```

```
#include <dht.h>
```

```
#include <LiquidCrystal.h>
```

```
dht DHT;
```

```
#define DHT11_PIN 4
```

```
LiquidCrystal lcd(6, 7, 5, 4, 3, 2);
```

```
void setup() {
```

```
  lcd.begin(16, 2);
```

```
}
```

```
void loop() {
```

```
  int chk = DHT.read11(DHT11_PIN);
```

```
  lcd.setCursor(0, 0);
```

```
  lcd.print("Temperature = ");
```

```
  lcd.print(DHT.temperature);
```

```
  lcd.setCursor(0, 1);
```

```
lcd.print("Humidity = ");  
lcd.print(DHT.humidity);  
  
delay(2000);  
}
```

• STEPPER MOTOR

```
#include <AccelStepper.h>  
  
// Define the motor interface type (4 for Half-step)  
#define HALFSTEP 4  
  
// Motor pin definitions  
#define motorPin1 13  
#define motorPin2 12  
#define motorPin3 11  
#define motorPin4 10  
  
// Initialize the stepper library  
// Note the pin sequence (1, 3, 2, 4) which is required for the ULN2003  
driver
```

```
AccelStepper stepper1(HALFSTEP, motorPin1, motorPin3, motorPin2,  
motorPin4);
```

```
void setup() {  
  stepper1.setMaxSpeed(1000.0);  
  stepper1.setAcceleration(1000.0);  
  stepper1.setSpeed(100);  
  
  // Set initial target position  
  stepper1.moveTo(-200);  
}
```

```
void loop() {  
  // Check if the motor has reached its target position  
  if (stepper1.distanceToGo() == 0) {  
    // Reverse direction by moving to the opposite coordinate  
    stepper1.moveTo(-stepper1.currentPosition());  
  }
```

```
  // Must be called as frequently as possible to move the motor  
  stepper1.run();  
  
  delay(1);
```



```
}
```

• DC MOTOR

```
#include <lpc214x.h>
```

```
void clock_wise(void);
```

```
void anti_clock_wise(void);
```

```
unsigned int j = 0;
```

```
unsigned int delay;
```

```
int main() {
```

```
    IODIR = 0x00000900; // Set P0.8 and P0.11 as output
```

```
    IOSET = 0x00000100; // Initial state
```

```
    while(1) {
```

```
        clock_wise();
```

```
        for(j = 0; j < 400000; j++); // Software delay
```

```
        anti_clock_wise();
```

```
        for(j = 0; j < 400000; j++); // Software delay
```

```
    }
```

```
}
```

```
void clock_wise(void) {  
    IOCLR = 0x00000100;    // Clear P0.8  
    for(int i = 0; i < 10000; i++);  
    IOSET = 0x00000100;    // Set P0.8  
}
```

```
void anti_clock_wise(void) {  
    IOCLR = 0x00000100;    // Clear P0.8  
    for(int i = 0; i < 10000; i++);  
    IOSET = 0x00000100;    // Set P0.8  
}
```

•Blinking

```
#include <lpc21xx.h>
```

```
unsigned int delay;
```

```
int main() {  
    PINSEL1 = 0x00000000; // Configure pins as GPIO
```

```
IODIR = 0x00FF0000; // Set P0.16 to P0.23 as output
```

```
while(1) {  
    // All LEDs OFF  
    IOCLR = 0x00FF0000;  
    for(delay = 0; delay < 500000; delay++);  
  
    // All LEDs ON  
    IOSET = 0x00FF0000;  
    for(delay = 0; delay < 500000; delay++);  
}  
}
```

● RTOS

```
#include <Arduino_FreeRTOS.h>
```

```
// Pin definitions
```

```
#define redled A5
```

```
#define greenled A4
```

```
// Custom type for profiling
```

```
typedef int TaskProfiler;
```

```

TaskProfiler redledprofiler;
TaskProfiler greenledprofiler;

// Function prototypes for tasks
void redledTask(void *pvParameters);
void greenledTask(void *pvParameters);

void setup() {
    Serial.begin(9600);
    while (!Serial); // Wait for serial port to connect

    // xTaskCreate(TaskFunction, Name, StackSize, Parameter,
    Priority, TaskHandle)
    xTaskCreate(redledTask, "redled Task", 128, NULL, 2, NULL);
    xTaskCreate(greenledTask, "greenled Task", 128, NULL, 1,
    NULL);
}

void loop() {
    // FreeRTOS handles task switching; loop remains empty.
}

/* --- Red LED Task --- */
void redledTask(void *pvParameters) {
    pinMode(redled, OUTPUT);

    while (1) {
        redledprofiler++;
        Serial.print("redled profiler : ");
        Serial.println(redledprofiler);
    }
}

```

```
    digitalWrite(redled, HIGH);
    vTaskDelay(200 / portTICK_PERIOD_MS); // FreeRTOS non-
blocking delay
    digitalWrite(redled, LOW);
    vTaskDelay(200 / portTICK_PERIOD_MS);
}
}
```

```
/* --- Green LED Task --- */
```

```
void greenledTask(void *pvParameters) {
    pinMode(greenled, OUTPUT);
```

```
    while (1) {
        greenledprofiler++;
        Serial.print("greenled profiler : ");
        Serial.println(greenledprofiler);
```

```
        digitalWrite(greenled, HIGH);
        vTaskDelay(500 / portTICK_PERIOD_MS); // FreeRTOS non-
blocking delay
        digitalWrite(greenled, LOW);
        vTaskDelay(500 / portTICK_PERIOD_MS);
    }
}
```